Recommendation Engine

Technical Design & Architecture Document

---

1. Summary

This document defines the architecture for building a contextual Recommendation Engine for super.money. The system recommends the next-best-product or next-best-action to users across all product lines: UPI, Cards, Loans, FD, Referrals etc.

A central insight driving this design is that super.money is a UPI-heavy app. Users open the app multiple times daily for UPI transactions, creating high-frequency micro-moments where they are financially engaged and contextually primed. The entire system is designed around exploiting these UPI moments to drive adoption of higher-value products.

The primary signal source is SMS insights (via Finbox), which provide a near-complete view of each user's financial life: income, balances, loans, credit cards, investments, and repayment behavior. This is augmented by super.money's own transactional data and, in later phases, app behavioral events from Clickhouse.

The system evolves across three phases: V0 (rule-based scoring), V1 (ML models + behavioral signals), and V2 (real-time, event-driven personalization).

2. Recommendation Philosophy & Objective Framework

2.1 The Fundamental Question

Before defining architecture or algorithms, the engine must answer: **what is it optimizing for?**

super.money's product portfolio contains two fundamentally different categories:

Category	Products	Optimizes For	Success Means
Monetization	Cards, Loans	Conversion, revenue, user spending	User transacted more, borrowed more
Financial Health	FD, debt reduction, savings/investment nudges	User surplus, financial wellness	User invested more, reduced debt, made better financial decisions

These categories require different scoring philosophies, different success metrics, and different ethical guardrails. A single propensity score that ranks a loan and an FD on the same axis is conceptually incoherent — it's like a fitness app ranking "order pizza" and "go for a run" on the same list because both have high engagement probability.

2.2 The Ethical Dimension

This is not just a product design question — it has real ethical stakes.

Consider the "credit-seeking intensity" signal: a user who installed 3 lending apps in 30 days and is applying for credit everywhere. Our scoring logic treats this as the strongest loan propensity signal. But this user could be:

- Profile A — The Rate Shopper: Financially stable, comparing offers for a planned purchase. Recommending a competitive loan here is genuinely helpful.
- Profile B — The Debt Spiraler: Over-leveraged, bouncing payments, desperately seeking any credit. Recommending another loan here is potentially predatory.

The same signal demands opposite recommendations depending on context. An engine optimizing purely for conversion would aggressively target Profile B (most likely to click "Apply Now"), while responsible recommendation would suppress credit products entirely.

Scenario	Pure Conversion Approach	Responsible Approach
High credit-seeking + high delinquency	Recommend loan (hot lead)	Suppress credit; nudge savings/debt mgmt
High CC utilization + rising EMI	Recommend new card (they want credit)	Suppress card; consider FD for emergency fund
Salary credit + zero savings + high spend	Recommend card (capture spends)	Recommend FD first (build buffer before spending)

2.3 Three Approaches

Option A: Single Engine with "Health First" Guardrails

One engine for all products, with a Financial Health Score as a policy gate. The engine optimizes for conversion but within ethical boundaries. Healthy users get full catalog. Neutral users get savings-boosted rankings. Stressed users get only health/savings products — credit is hard-suppressed.

Pros	Cons
One engine, one infrastructure	"Financial health score" thresholds are hard to calibrate
Ethical guardrails are explicit and auditable	Business teams may resist suppression of high-propensity leads
Simple to implement (policy layer on top of scoring)	Binary health gates can feel crude for grey-zone users
Strong brand narrative: "we recommend what's right"	

Option B: Two Separate Recommendation Tracks

Track 1 "Grow" (Cards, Loans, Referrals) optimizes for conversion and revenue. Track 2 "Thrive" (FD, savings) optimizes for user wellness. Slot allocation between tracks varies by user health (stressed = 80% Thrive).

Pros	Cons
Each track independently optimized with own metrics	Two engines, more engineering complexity
Clear organizational ownership per track	Arbitration between tracks is a meta-optimization problem
No philosophical conflict within a single scorer	Slot allocation becomes a political decision
Easier to measure each objective independently	Mixed signals (FD reco followed by card reco)

Option C: Financial Health as the North Star

Every recommendation is evaluated on whether it makes the user financially better off. FD improves health. Card is eligible only if user won't overspend. Loan only if it consolidates higher-interest debt. Revenue is a byproduct of genuine improvement.

Pros	Cons
Strongest brand differentiator	"Health improvement" is a 6-12 month trailing metric — very hard to optimize
Naturally prevents predatory patterns	Severely limits credit recommendations short-term
Builds deep user trust and retention	Requires causal inference to measure actual impact
Regulatory-friendly (ahead of future fintech regulation)	Too restrictive for early-stage business model

2.4 Proposed Approach for super.money

We recommend Option A — Single Engine with "Health First" Guardrails — as the starting point, with elements of Option B informing how we measure success.

The reasoning:

- Option C is aspirational but premature. Measuring health improvement requires 6-12 months of longitudinal data and causal inference capabilities that don't exist yet. It can be a V2/V3 north star.
- Option B adds significant complexity (two pipelines, two optimization loops, meta-arbitration) that is overkill at this stage.

- Option A gives ethical guardrails without the complexity overhead. One engine, one Financial Health Score as a policy gate, clear rules for when credit is suppressed.

2.5 Open Questions for Discussion

The following questions should be resolved with product management and leadership before finalizing:

- 1. Where to draw health score thresholds? What obligations-to-income ratio defines "stressed"? Is 50% the right line? Should a single bounce in 90d trigger suppression, or does it need to be a pattern?
- 2. Hard gate vs soft penalty? Current recommendation is hard suppression for Stressed users. Alternative: heavy scoring penalty (-50 points) that makes credit unlikely but not impossible. Hard gates are more defensible ethically; soft penalties are more commercially flexible.
- 3. How to handle explicit user intent? If a Stressed user navigates to the Loans section (clear intent), do we still suppress? Does explicit intent override the health gate?
- 4. Should we disclose the suppression? "We’re not showing you a loan right now because we think it’s not the right time." Transparency builds trust but may frustrate users.
- 5. Revenue impact tolerance? Suppressing credit for Stressed users will reduce short-term conversion. What drop is acceptable for ethical positioning?

3. Problem Statement & Goals

3.1 Problem

super.money operates multiple product lines but lacks a unified system to determine which product to recommend to which user, at what time, on which surface. Current product adoption is driven by user self-discovery, broadcast campaigns, and manual segmentation — resulting in low conversion, user fatigue, and missed cross-sell opportunities.

3.2 Goals

- Increase product adoption by delivering contextually relevant, personalized recommendations.
- Build scalable recommendation infrastructure that supports all current and future product lines.
- Maintain clear separation between feature computation, scoring, and delivery.
- Leverage UPI as the primary engagement loop and distribution channel for all other products.

- Ensure recommendations are ethically sound — suppress credit products for financially stressed users (per Section 2).

3.3 Success Metrics

Current State: No systematic recommendation system exists. Organic product adoption rates will be baselined during V0 rollout (first 2 weeks at 10% traffic serve as control group).

Monetization Products (Card, Loan):

Metric	Definition	V0 Target
Recommendation CTR	% users who click a recommendation	>15%
Conversion Rate	% of clicks that complete product journey	>3%
Revenue per Reco Served	Revenue attributed to served recommendations	Track baseline in V0
Lift over Organic	Reco-driven conversion vs organic adoption rate	>2x

Financial Health Products (FD, Savings, Invest):

Metric	Definition	V0 Target
Adoption Rate	% of FD recommendations resulting in FD opened	>5%
Surplus Growth (30d)	Avg balance change 30d post-FD recommendation	Positive
Health Trend (90d)	% users whose health_tier improved or stayed stable	>80%

System Metrics:

Metric	Definition	V0 Target
Coverage	% active users receiving at least one recommendation	>70%
Fatigue Index	Avg recommendations per user per week	<5
API Latency (P99)	Recommendation serving response time	<50ms
Suppression Rate	% users where credit products were health-suppressed	Track (awareness metric)

4. Data Landscape

4.1 Data Sources

All data sources are ingested into the Feature Store. The Recommendation Processor never reads raw data from BigQuery, internal DBs, or Clickhouse directly — it only consumes enriched features from the Feature Store. This makes the Feature Store the single point of consumption for all downstream systems (recommendations, risk, marketing).

Source	Ingestion Path	Freshness	Key Signals	Phase
SMS Insights (Finbox)	Finbox → BigQuery → Feature Store Flink	Periodic push	Income, balances, loans, CC, investments, bills, delinquencies	V0+
Internal Transactions & Product Data	Internal Data Warehouse → Feature Store Flink	Flink batch pull	super.money product holdings, UPI txn history, KYC status, card data	V0+
App Activity Events	Clickhouse → Feature Store Flink (aggregated)	Flink pulls aggregates, not raw events	Session frequency, page views, search queries, funnel drop-off	V1+
Journey System State	Journey DB → Feature Store Flink	Flink batch pull	Past recommendations shown, delivery history, user response	V0+ (read)

4.2 Current Pipeline

All sources → Feature Store Flink Processor → Enriched features in Offline Feature Store → Feature Store API (for online serving to other consumers).

The Recommendation Processor reads from the same Offline Feature Store in batch.

4.3 Key Feature Categories from SMS Insights

Category	Examples	Recommendation Use
Income & Salary	calculated_income, salary_m1-m3, salary_date	Eligibility gating, income trends for upsell
Bank Balances	avg_balance, min/max_bal across time windows	FD propensity, financial health

Credit Cards	cc_count, utilization, credit_limit, cc_applications	Card recommendation, credit need detection
Loans & EMI	loan_count, EMI amounts, obligations, lenders	Loan refinance, debt burden assessment
Delinquency & Bounces	delinquency counts, cheque/ECS/SI bounces	Risk gating, product exclusion
Investments	fd_flag, mf_flag, equity_flag, insurance_flag	FD/MF recommendation, investment readiness
Digital Payments	UPI/PhonePe/Paytm/Amazon flags & vintage	UPI activation strategy, competitive positioning
Risk & Fraud	fraud_flag, fake GPS apps, lending app count	Hard exclusion filter
Affordability Scores	fis_affordability, fps_investment_probability	Pre-built propensity signals from Finbox

4.4 Derived Features (for Recommendation Engine)

Feature	Logic	Purpose
disposable_income_ratio	(income - obligations - avg_debits) / income	Financial headroom for new products
income_stability_score	CV of salary across 3 months	Confidence in income-based eligibility
credit_seeking_intensity	lending_apps_c30 + loan_apps_c30 + cc_apps_c30	Aggregated credit demand signal
debt_burden_ratio	total_EMI / income	Debt service capacity
surplus_cash_indicator	avg_balance - obligations - (avg_debits * 1.2)	FD/investment trigger
financial_stress_score	Composite of bounces, delinquencies, declining balances	Risk-adjusted recommendation gating
salary_timing_flag	Days since last salary credit	Timing optimization
financial_health_score	Composite of: balance trend (declining = negative), bounce count in 90d, delinquency count, obligations-to-income ratio, lending app install velocity	Policy gate for product eligibility (Section 2)
health_tier	Bucketed from financial_health_score: Healthy / Neutral / Stressed	Direct input to Health Policy Gate

5. UPI-Centric Recommendation Strategy

5.1 UPI as the Primary Engagement Loop

super.money is fundamentally a UPI-heavy app. Users open the app 3-4 times daily for payments, creating a usage pattern fundamentally different from lending or investment apps. This has four implications for recommendations:

- High frequency touchpoints: Multiple daily opportunities to surface recommendations.
- Transactional context: Every UPI transaction carries amount, merchant category, and recipient type — powerful real-time signals.
- Financial mindset: Users are already thinking about money, making them receptive to adjacent product offers.
- Trust and habit: Regular UPI usage builds familiarity that is the foundation for cross-selling higher-commitment products.

5.2 The Transaction Moment Advantage

The post-UPI transaction screen is the single most valuable recommendation surface — the fintech equivalent of Amazon's "Frequently Bought Together" on the cart page. At this moment, the user has demonstrated financial intent, feels completion satisfaction, and hasn't yet switched context.

Beyond post-transaction, fintech has additional high-value moments:

Moment	User State	Why It Converts	Best Products
Post-UPI Transaction	Task done, still in app	Highest attention window	Card (P2M), FD (high bal), Referral (P2P)
Salary Credit (SMS)	Feeling flush	Peak investment willingness	FD, Card activation
Failed Transaction	Frustrated, needs alternative	High motivation	Credit line, Card
Bill Payment Done	Responsible mindset	Primed for planning	Autopay, Card (bill rewards)
Low Balance Detected	Anxious	Problem-aware	Loan, Credit line (health-gated)
Cashback Redemeed	Delighted	Promoter of the app	Referral
First Txn of Day	Fresh session, open to discovery	Pre-task attention	Top-ranked from engine

Note: For Low Balance and Failed Transaction moments, the Health Policy Gate (Section 2.4) applies. Credit products are only recommended to Healthy/Neutral users. Stressed users see savings/health nudges instead.

5.3 Product Graduation Ladder

Users graduate through products anchored by UPI engagement. High-commitment products are gated behind UPI trust signals — never recommend a loan to a user who hasn't built a UPI habit first.

Stage	Behavior	Cross-Sell Target	Gate
1. UPI Activated	First few transactions	Referral	3+ txns in 7 days
2. Regular User	10+ txns/month	Credit Card	P2M spends > 5K/mo + no card
3. High-Value	Large spends, bill payments	FD	Avg balance > 50K + salary + no FD
4. Multi-Product	Uses UPI + Card or FD	Personal Loan	Clean repayment + 6mo vintage
5. Power User	Deep engagement	Premium + Referral	High LTV + social indicators

6. Recommendation Surfaces & Placement

6.1 E-Commerce Analogy

E-Commerce	E-Commerce Reco	Fintech Surface	Fintech Reco
Cart / Checkout	Add-ons, bundles	Post-UPI Transaction/Cashback Redemption	"Pay with card, earn 2% back"
Homepage	Personalized picks	App Home Feed	Top-ranked recommendation
Order Confirmation	Related products	Transaction History/Spend Insights	"Spent 15K on dining — get 5% back"
Search Results	Sponsored products	Pre-Payment Screen	"Pay via card for rewards"
Wishlist (price drop)	Price drop alerts	Salary Credit (push)	"Salary arrived — park 20K in FD"
Empty Cart	Popular products	Dormant User	"Welcome back! Start earning with FD"

6.2 Surface Specifications

Each surface has distinct user intent, attention window, and conversion profile:

Surface	Trigger	Format	Expected CTR	Fatigue Limit

Post-UPI Transaction	UPI txn success	Compact card below confirmation	8-15%	50% of txns (alternate)
App Home Feed	App opened	Card carousel (1-3 recos)	3-6%	Rotate across sessions
Transaction History	User views passbook	Inline insight card	5-8%	1 per view, refresh weekly
Pre-Payment Screen	About to pay (amt entered)	One-line nudge	10-20% (method switch)	P2M > ₹200 only
Salary Credit Push	SMS salary detection	Rich push notification	4-7%	1 per month
Failed Txn / Low Bal	Txn failure or low balance	Contextual banner	6-10%	1 per event, 48hr cooldown
Bill Payment Flow	Bill payment complete	Post-payment card	5-8%	After first manual pay per biller

6.3 Surface-Product Affinity Matrix

Not every product should appear on every surface. The Serving API uses this matrix as a filter. The surface_affinity_multiplier in the ranking formula (Section 10) translates these ratings into numerical adjustments: BEST=1.0, GOOD=0.8, MEDIUM=0.5, LOW=0.1.

Surface	FD	Card	Loan	Referral
Post-UPI Transaction	GOOD	BEST	LOW	GOOD
App Home Feed	BEST	BEST	GOOD	MEDIUM
Transaction History	GOOD	BEST	MEDIUM	LOW
Pre-Payment Screen	LOW	BEST	LOW	LOW
Salary Credit Push	BEST	GOOD	MEDIUM	LOW
Failed Txn / Low Bal	LOW	GOOD	BEST	LOW
Bill Payment Flow	LOW	BEST	LOW	LOW

This matrix is a V0 hypothesis based on first-principles analysis of user intent per surface. It will be validated against actual surface-level conversion data during V0 rollout and refined iteratively. By V1, qualitative ratings will be replaced with data-driven affinity scores derived from observed CTR and conversion rates per surface-product combination. By V2, it is replaced entirely by ML-driven surface selection.

6.4 Messaging Principles by Surface

Surface	Principle	Tone	Example (FD)
Post-UPI Txn	Brief + contextual (user leaving)	Casual	"Your 50K balance could earn 3,750/yr"

Home Feed	Value prop + personalized numbers	Confident	"Park 1.2L in FD. Earn 9,000 at 7.5%"
Txn History	Insight-driven ("did you know")	Analytical	"80K+ avg balance — that's 500/mo in FD"
Salary Push	Timely + actionable	Urgent, friendly	"55K salary arrived! Park 20K before you spend it"
Failed Txn	Empathetic + solution	Supportive	N/A for FD (wrong surface)

7. High-Level Architecture

7.1 Four-Layer Architecture

The system is decomposed into four layers with clear boundaries:

Layer	Component	Responsibility	Status
1. Feature Store	Flink + Offline Store (existing)	Product-agnostic user profile from ALL data sources (single point of consumption)	Existing (extend)
2. Reco Processor	Separate Flink job (new)	Health policy gate → eligibility → scoring → ranking → write recommendations	New
3. Serving API	Microservice (new)	Surface filtering, fatigue checks, txn_context, A/B assignment	New
4. Delivery	Journey System (existing)	Multi-touch delivery, tracks response, channel fatigue	Existing

7.2 Data Flow

V0/V1 (Batch):

All Data Sources → Feature Store Flink → Offline Feature Store → Recommendation Flink (reads batch from offline store) → Recommendation Store → Recommendation Serving API → Journey System / PN.

V2 (Hybrid Batch + Streaming):

Same as above, plus: Event triggers (Finbox push, Clickhouse events) → Streaming Feature Store path → Real-time feature updates → Streaming Recommendation Processor for immediate re-scoring of triggered users.

Note: In V0/V1, the Recommendation Flink reads from the Offline Feature Store in batch mode. There is no real-time Bigtable read requirement — the processor scans features written by the Feature Store Flink. This is simpler, cheaper, and avoids real-time consistency concerns. The real-time path is introduced only in V2 when event-driven scoring is added.

7.3 Key Architectural Principles

- Single point of consumption: All data sources feed into the Feature Store. The Recommendation Processor reads only from the Feature Store, never from raw sources.
- Health-first policy: The Health Policy Gate is a first-class architectural component in the Reco Processor, not an afterthought filter.
- Pre-compute over real-time (V0/V1): Recommendations batch-computed and cached. Serving API is a thin read layer.
- Configuration-driven: Scoring rules, health thresholds, affinity matrix, business priorities — all externalized in config.
- Surface-aware serving: API accepts surface + txn_context parameters, applies affinity matrix and surface-specific fatigue.

8. Component Design Overview

8.1 Feature Store (Existing)

Ingests all data sources via Flink. Writes enriched features to the Offline Feature Store. For the recommendation engine, additionally computes: financial_health_score, health_tier, and the derived features listed in Section 4.4.

8.2 Recommendation Processor (New)

A separate Flink job with a six-stage pipeline per user:

- Stage 1 — Candidate Generation: All products minus what user already holds on super.money.
- Stage 2 — Health Policy Gate: Apply health_tier to filter/adjust candidates. Stressed: remove Cards, Loans, BNPL. Neutral: flag for scoring adjustment. Healthy: pass through.
- Stage 3 — Eligibility Filtering: Binary pass/fail checks (min income, no fraud, data sufficiency).

- Stage 4 — Propensity Scoring: 0-100 score per product (Section 9). For Neutral users: +15 boost for health products, -10 penalty for credit products.
- Stage 5 — Ranking & Arbitration: Multi-factor ranking (Section 10).
- Stage 6 — Write: Top 5 ranked recommendations per user to Recommendation Store.

The Health Policy Gate runs before scoring, not after ranking. This is critical — it prevents wasting compute on products that will be suppressed, and ensures the ranking algorithm is never influenced by suppressed candidates.

Trigger event detection (changes between runs):

Trigger	Detection	Action
Salary credit	salary_m1 changed	Prioritize FD within 24hrs
Credit-seeking spike	lending_apps_c30 increased by ≥2	Prioritize loan (Healthy/Neutral only)
Balance surge	latest_balance up >50% vs avg	Trigger FD (idle money)
CC app elsewhere	cnt_cc_applications_c30 up	Prioritize card (Healthy/Neutral only)
EMI bounce	ecs_bounce or si_bounce appeared	Suppress credit; may downgrade health_tier
Health tier change	health_tier worsened	Re-run full pipeline; suppress newly ineligible products

8.3 Recommendation Serving API (New)

Thin, low-latency service (sub ms P99) that reads pre-computed recommendations and applies serving-time logic:

- Surface filtering: Applies the surface-product affinity matrix to re-rank based on the surface parameter.
- Fatigue enforcement: Checks delivery history before returning recommendations.
- A/B experiment assignment: Deterministic hash-based bucketing for scoring and creative variants.
- Returns a ranked list (not single recommendation) to give the flexibility.

8.4 Journey System Integration

The Journey system is the delivery layer — it does not make recommendation decisions. It receives the top recommendation from the Serving API, manages the delivery sequence (in-app card → push → reminder → expire), tracks engagement (impression, click, dismiss, convert), and feeds data back for analytics. Integration is one-directional in V0 (engine → journey), becoming two-way in V1+ with feedback loops.

9. Product Scoring Logic

Each product has eligibility checks (binary pass/fail) and propensity scoring (weighted features, 0-100). V0 is rule-based. V1 replaces with ML model predictions. All credit product scores are subject to the Health Policy Gate (Section 8.2).

9.1 Fixed Deposit (FD)

Signal	Wt	Logic
Avg balance (30d)	25	≥1L: 25, ≥50K: 20, ≥25K: 10
Surplus cash indicator	25	≥50K: 25, ≥20K: 15, else: 5
Income stability	15	CV<0.1: 15, CV<0.2: 10, else: 5
No delinquencies (90d)	10	Zero: 10, else: 0
Salary timing	10	Within 3d of credit: 10, 7d: 5
Investment propensity	10	fps_investment>0.7: 10, >0.5: 5
No bounces	5	All zero: 5, else: 0

Eligibility: No existing FD, income ≥15K, no fraud, data sufficient. Threshold: ≥40. Neutral-tier users receive +15 boost to final score.

9.2 Personal Loan

Signal	Wt	Logic
Credit-seeking intensity	30	lending_apps≥3: 30, ≥1: 20, loan_apps_c60>0: 10
Income level	20	≥50K: 20, ≥30K: 15, ≥20K: 10
Clean repayment (90d)	15	Zero delinquency + no bounces: 15
Disposable income ratio	15	>0.4: 15, >0.25: 10, else: 5
Prior loan experience	10	Closed loans >0 (repaid): 10
Balance stability	10	min_bal_c30 > 5K: 10

Eligibility: Income ≥20K, affordability above threshold, debt burden <0.5, no delinquency c30, no fraud. Threshold: ≥50. Health gate: Healthy tier only. Neutral-tier users receive -10 penalty. Stressed-tier users are excluded before scoring.

9.3 Credit Card

Signal	Wt	Logic
No CC + good income	25	cc_user=false AND income≥30K: 25, else: 10
High CC utilization elsewhere	25	>80%: 25, >60%: 15

CC applications elsewhere	20	cc_apps_c30>0: 20, c60>0: 10
Digital savviness	15	high: 15, medium: 10, else: 5
Income level	15	≥50K: 15, ≥35K: 10, ≥25K: 5

Eligibility: No super.money card, income ≥25K, no fraud, DPD<30. Threshold: ≥45. Health gate: Healthy or Neutral tier. Neutral-tier users receive -10 penalty. Stressed-tier users are excluded before scoring.

9.4 UPI Activation

Signal	Wt	Logic
Existing UPI user elsewhere	30	upi_flag=true + high volume: 30, moderate: 20, new: 10
Digital payment activity	25	ewallets≥2: 25, ≥1: 15, else: 5
Transaction frequency	20	debit_txn_c30>20: 20, >10: 15, else: 5
Competitor platform user	15	phonepe/paytm=true: 15, else: 5
App engagement (V1)	10	Reserved for Clickhouse signals

No health gate — UPI activation is available to all health tiers.

9.5 Referral

Signal	Wt	Logic
Digital savviness	25	high: 25, medium: 15, else: 5
Social/communication apps	25	count>10: 25, >5: 15, else: 5
Active super.money user	20	txn_c30>5: 20, >2: 10
Multi-product holder	15	≥2 products: 15, 1: 5
P2P activity (network proxy)	15	High UPI P2P debits: 15

No health gate — referral is available to all health tiers.

10. Ranking, Arbitration & Fatigue

10.1 Ranking Formula

final_score = ((propensity + health_adjustment) × 0.50) + (business_priority × 0.25 × 100) - (fatigue_penalty × 0.15) + (trigger_boost × 0.10), then multiplied by surface_affinity_multiplier.

Where health_adjustment = +15 for savings products if Neutral tier, -10 for credit products if Neutral tier, 0 for Healthy tier. Stressed-tier credit products already removed at the Health Policy Gate before scoring.

Surface affinity multiplier (applied at serving time): BEST=1.0, GOOD=0.8, MEDIUM=0.5, LOW=0.1.

10.2 Business Priority

Product	Priority	Rationale
Card	0.9	Highest strategic priority for retention
FD	0.8	High margin, key growth target
Loan	0.7	Revenue driver, risk-adjusted
UPI	0.5	Engagement driver, lower direct monetization
Referral	0.4	Always-on, lower priority

10.3 Fatigue Rules

Rule	Constraint	Effect
Same product cooldown	Shown in last 48 hours	+30 penalty
Dismissed cooldown	User dismissed	+50 penalty, 7-day cooldown
Daily limit	Max 3 recos/day	Block beyond limit
Weekly limit	Max 10 recos/week	Block beyond limit
Converted exclusion	Already converted	Remove permanently

11. Algorithm & ML Strategy

The recommendation system evolves through three algorithmic phases, each building on the data generated by the previous phase. The design principle is: start with interpretable rules that generate labeled training data, then graduate to ML when you have enough conversion signal to train meaningful models.

11.1 V0: Rule-Based Scoring Engine

The V0 algorithm is a deterministic, config-driven weighted scoring system. Each product has a set of rules that map feature values to partial scores, which are summed to produce a propensity score (0-100).

How it works:

- Each rule is a (feature, threshold, score) tuple. For example: if avg_balance_c30 >= 1L, add 25 points to FD score.
- Rules are grouped by product and stored in external YAML/JSON config. No code changes needed to tune thresholds.
- Eligibility rules are hard gates (binary pass/fail). Propensity rules are additive (weighted sum).
- Conflict resolution: if a user qualifies for multiple products, the ranking formula (Section 9) arbitrates.

Why rules first, not ML:

- No training data exists yet. You cannot train a conversion model without historical conversion events.
- Rules are fully interpretable. When a recommendation looks wrong, you can trace exactly which rule caused it.
- V0 generates the training data for V1. Every serve, impression, click, and conversion is logged with the full feature vector, creating labeled examples for future ML models.
- Rules are fast to iterate. Product managers can adjust thresholds in config and see results in the next Flink cycle.

Limitations:

- Cannot capture complex, non-linear feature interactions (e.g., "high income + high CC utilization + recently installed lending apps" as a combined signal).
- Scoring weights are manually assigned, not optimized against actual conversion data.
- No personalization beyond segment-level rules — two users with the same feature profile get the same score.

11.2 V1: ML Driven

Once V0 has generated 2-3 months of recommendation events (serves, clicks, conversions), train ML models (XGBoost/LightGBM or Collaborative Filtering[like GPay] ?). **ML model choice is an open decision for now and not the scope of this document**.

Model deployment:

- Shadow mode first: run ML scoring alongside rules for 2-4 weeks. Log both scores, serve rules. Compare offline which would have performed better.
- A/B test: randomly assign 50% of users to ML-scored recommendations, 50% to rule-based. Measure conversion lift.

- Graduate: if ML outperforms rules by a statistically significant margin, promote to production.
- Fallback: rules remain as a fallback for cold-start users or when model confidence is low.

Feature engineering for ML (beyond raw Finbox attributes):

- Velocity features: change in balance/income/EMI over 30/60/90 days (trend direction matters more than absolute value).
- Ratio features: debit-to-credit ratio, EMI-to-income ratio, utilization-to-limit ratio.
- Recency features: days since last salary, days since last loan disbursement, days since last app open.
- Clickhouse behavioral features (V1 addition): pages_visited_c7, product_page_views_c30, search_queries, session_frequency, funnel_drop_off_stage.

11.3 V2: Contextual & Real-Time Models

V2 introduces three algorithmic capabilities:

A. Explore/Exploit

Problem: V0 and V1 only exploit (recommend what the model is most confident about). This creates a feedback loop — products that were never recommended never get conversion data, so the model never learns about them. New product lines (e.g., insurance, mutual funds) would never get exposure.

B. Real-Time Re-Scoring at Serving Time

In V0/V1, recommendations are fully pre-computed in batch. In V2, the Serving API makes a lightweight real-time call to the Feature Store API to fetch fresh features (especially Clickhouse session features) and re-scores the pre-computed candidate list using a fast online model. This enables:

- Session-aware recommendations: "user has viewed the FD page 3 times this session" boosts FD score in real-time.
- Transaction-level personalization: "user just paid 8K at BigBasket" generates a grocery-card creative immediately.
- Recency-sensitive scoring: "user was active 2 minutes ago" vs "user hasn’t opened app in 3 days" affects ranking.

C. Sequence Models for Product Adoption Prediction

Once you have 6-12 months of multi-product adoption data, you can model the product adoption sequence:

- Given a user’s current product portfolio (e.g., UPI + Card), predict the most likely next product (e.g., FD).
- This replaces the manual graduation ladder (Section 5.3) with data-driven sequencing.

11.4 Cold Start Handling

New users with insufficient SMS data require a fallback strategy:

Scenario	Approach	Default Recommendation
No SMS data yet (Finbox pending)	Popularity-based: recommend what converts best for the overall population	UPI activation (low commitment, builds engagement)
Partial SMS data (<30 days)	Conservative rules with higher thresholds; only recommend if confidence is high	UPI + Referral (both low-risk)
Full SMS data, no super.money history	Full scoring pipeline with SMS features; treat as new user lifecycle stage	Score-based (SMS gives enough signal)
Returning user, stale features	Serve last known recommendations with reduced confidence; trigger feature refresh	Previous top recommendation with "stale" flag

12. Iterative Roadmap (V0 / V1 / V2)

12.1 V0 — Rule-Based Foundation

Goal: First working engine with rule-based scoring, health policy gate, 3 primary surfaces.

Deliverable
Recommendation Store design, Flink skeleton, config service, surface-product affinity matrix, Financial Health Score feature in Feature Store
Scoring rules for 5 products, derived features, Health Policy Gate, UPI graduation gates, health threshold calibration with product team
Surface-aware Serving API, post-UPI txn + home feed + salary push surfaces, Feedback API with surface attribution
Journey integration, monitoring (per-surface CTR, suppression rate, split metrics), rollout 10% (baseline) → 50% → 100%

Not in V0: Clickhouse integration, ML models, real-time scoring, advanced A/B testing.

12.2 V1 — ML + Behavioral Signals

Goal: Replace rules with ML models, add Clickhouse behavioral signals, launch all 7 surfaces.

Deliverable
ML training pipeline, initial models (health-tier-aware training data), txn_history + pre_payment surfaces
Clickhouse aggregated features in Feature Store, behavioral feature engineering, UPI graduation refinement
ML shadow mode, failed_txn + bill_payment surfaces, measure health suppression impact on revenue vs retention
A/B test: ML vs rules, surface-level creative A/B tests
Graduate winning models, full feedback loop, data-driven affinity matrix

---

12.3 V2 — Real-Time & Advanced Personalization

Goal: Event-driven scoring, explore/exploit algorithm, transaction-level personalization.

13. Key Decisions & Trade-offs

Decision	Choice	Alternative	Rationale
Recommendation philosophy	Single engine + Health First guardrails (Option A)	Two tracks (B) or Health north star (C)	Ethical guardrails without complexity overhead; Option C as future aspiration
Health Policy Gate placement	Before scoring (Stage 2 in pipeline)	After ranking (post-hoc filter)	Avoids wasted compute; ensures ranking isn't influenced by suppressed products
Feature Store as single data source	All sources ingest into Feature Store; Reco reads only from there	Reco reads from multiple raw sources	Single point of consumption; clean separation; enables multiple consumers
Reco Processor separate from Feature Store	Separate Flink job + storage	Embed in Feature Store	Independent deploy cycles, multiple consumers, cleaner debugging
Batch offline store for V0/V1	Reco Flink reads from offline store in batch	Real-time Bigtable reads	Simpler, cheaper; real-time not needed until V2 event-driven scoring

Rules before ML	Config-driven rules for V0	ML from day one	No training data; rules generate labels for V1; fully interpretable
SMS insights as primary signal	Finbox features drive V0	App activity as primary	SMS covers full financial life; app data thin for new app
Surface affinity matrix as V0 hypothesis	First-principles based, data-refined in V1	No surface filtering	Prevents mismatches; validated by data over time
UPI engagement as trust gate	Gate credit behind UPI history + health tier	Recommend all immediately	Builds trust before high-commitment asks

14. Risks & Mitigations

Risk	Severity	Mitigation
Health score miscalibration (too many users Stressed, excessive suppression)	High	Start conservative (only clearly stressed); monitor suppression rate; iterate thresholds with product team
Revenue impact from credit suppression	Medium	Track suppression rate and lost revenue separately; present as "cost of responsible lending"
Finbox data quality (parsing errors, missing fields)	High	Data quality checks in Feature Store; min data_sufficiency threshold
Finbox pipeline delay	Medium	Monitor latency; serve stale recos with lower confidence
Recommendation fatigue	High	Conservative fatigue limits; monitor dismiss rates; kill-switch per product/segment
Low conversion despite high CTR	Medium	Track full funnel (serve→click→convert); tune on conversion, not CTR
Cold start (new users, no SMS)	Medium	Popularity fallback; default to UPI + referral
Privacy / compliance risk	Critical	Ensure consent covers reco use; no PII in payloads; audit access
Surface cannibalization	Medium	Cross-surface impression caps; measure multi-surface impact